

```

1  """
2  Builds the processed feature set for the Smart Patio
  Shield models.
3
4  Reads raw weather + ONI data, merges them, engineers
  features and
5  builds the target, then writes a single processed
  parquet that all
6  downstream modeling consumes.
7  """
8
9  import hashlib
10 import json
11 from datetime import datetime, timezone
12 from pathlib import Path
13
14 import pandas as pd
15 import numpy as np
16
17 PROJECT_ROOT = Path(__file__).resolve().parents[2]
18 RAW_DIR = PROJECT_ROOT / "data" / "raw"
19 EXTERNAL_DIR = PROJECT_ROOT / "data" / "external"
20 PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
21
22 WEATHER_FILE = "weather_jamaica_2021-05-01_to_2026-05-01.parquet"
23 ONI_FILE = "noaa_oni_monthly.parquet"
24 OUTPUT_FILE = "patio_features.parquet"
25
26 # Wet-veranda target thresholds: provisional, see
  01_weather_eda.ipynb Section 12.
27 # Rule: light rain combined with gusty wind blows
  rain sideways through the grille;
28 # heavier rain reaches the patio regardless of wind.
  The thresholds were physically
29 # reasoned but are not yet empirically calibrated.
  EDA threshold-sensitivity analysis
30 # identified the light-rain precip threshold as the
  dominant lever, and the gust
31 # threshold (currently low relative to typical rainy-
  hour gusts) as the priority for

```

```

32 # recalibration.
33
34 WET_VERANDA_LIGHT_PRECIP_MM = 0.1
35 WET_VERANDA_GUST_KMH = 15
36 WET_VERANDA_HEAVY_PRECIP_MM = 1.0
37
38
39 def load_raw_data():
40     """
41     Load raw weather and ONI datasets, applying EDA
42     cleaning decisions.
43     EDA decisions (see 01_weather_eda.ipynb, sections
44     4 and 6):
45     - boundary_layer_height: 6-month coverage gap (
46       Dec 2023 - Jun 2024), cannot be reliably imputed.
47     - rain: a literal duplicate of precipitation (
48       Jamaica has no snow, and the archive API folds shower
49       into both). Correlation 1.0, zero diff.
50     """
51     weather = pd.read_parquet(RAW_DIR / WEATHER_FILE)
52     oni = pd.read_parquet(EXTERNAL_DIR / ONI_FILE)
53
54     drop_cols = ['boundary_layer_height', 'rain']
55     weather = weather.drop(columns = [c for c in
56     drop_cols if c in weather.columns])
57
58     return weather, oni
59
60 def merge_oni(weather: pd.DataFrame, oni: pd.
61 DataFrame) -> pd.DataFrame:
62     """Merge monthly ONI into hourly weather; forward
63     -fill trailing months."""
64     weather = weather.copy()
65     weather["year_month"] = weather["time"].dt.
66     to_period("M").dt.to_timestamp()
67
68     oni_cols = oni[["year_month", "oni", "enso_phase"]

```

```

64 ]]
65     df = weather.merge(oni_cols, on="year_month",
66         how="left")
67     # Forward-fill the trailing months (Apr/May 2026
) not yet published.
68     df = df.sort_values(["location", "time"]).
        reset_index(drop=True)
69     df["oni"] = df.groupby("location")["oni"].ffill
        ()
70     df["enso_phase"] = df.groupby("location")["
        enso_phase"].ffill()
71
72     return df
73
74
75 def add_temporal_features(df: pd.DataFrame) -> pd.
        DataFrame:
76     """
77     Add time-based features derived from the
timestamp.
78
79     Includes both raw integer encodings and cyclical
sin/cos
80     encodings (which preserve wraparound adjacency:
hour 23 sits next to
81     hour 0, December next to January). Day-of-week
is deliberately omitted:
82     the EDA confirms rain rate is flat across
weekdays, as it must be - the
83     atmosphere has no concept of the work week.
84     """
85
86     df = df.copy()
87
88     df['hour'] = df['time'].dt.hour
89     df['month'] = df['time'].dt.month
90
91     # Cyclical encodings: map each periodic value
onto the unit circle
92     df['hour_sin'] = np.sin(2 * np.pi * df['hour']

```

```

92 ] / 24)
93     df['hour_cos'] = np.cos(2 * np.pi * df['hour'
    ] / 24)
94     df['month_sin'] = np.sin(2 * np.pi * df['month'
    ] / 12)
95     df['month_cos'] = np.cos(2 * np.pi * df['month'
    ] / 12)
96
97     return df
98
99
100 def add_lag_features(df: pd.DataFrame) -> pd.
    DataFrame:
101     """
102         Add lag and trend features that capture recent
    atmosphere state.
103
104         Lag features look backward in time, exposing
    momentum and trends that
105         instantaneous snapshots miss (e.g., "pressure
    has been falling for 3 hours"
106         is a classic storm precursor invisible in any
    single hour's reading).
107
108         All operations are grouped by location so one
    city's history never leaks
109         into another's. Rows at the start of each
    location's series will be NaN
110         for the longer lags (up to 6 per location for
    the 6-hour pressure trend).
111         I'll leave these as NaN rather than fill -
    XGBoost handles missing values
112         natively, and downstream code can choose a fill
    strategy if needed.
113
114         Leakage note: 'precipitation' and '
    wind_gusts_10m' are components of the
115         'wet_veranda' target. Their rolling features are
    computed on shifted
116         values so the current hour is excluded -
    otherwise we'd be using the

```

```

117     target's own ingredients as features. Non-
118     targeted component variables
119     (pressure, humidity, clouds) have no such
120     constraints.
121     """
122
123     df = df.sort_values(['location', 'time']).
124     reset_index(drop=True).copy()
125
126     # Precipitation (target component) - past only,
127     # current hour excluded
128     df['precip_lag1'] = df.groupby('location')['
129     precipitation'].shift(1)
130     df['precip_lag3'] = df.groupby('location')['
131     precipitation'].shift(3)
132     df['precip_sum_3h'] = df.groupby('location')['
133     precipitation'].transform(
134     lambda x: x.shift(1).rolling(3, min_periods=
135     3).sum()
136     )
137
138     # Wind gust (target component) - past only,
139     # current hour excluded
140     df['gust_lag1'] = df.groupby('location')['
141     wind_gusts_10m'].shift(1)
142     df['gust_max_3h'] = df.groupby('location')['
143     wind_gusts_10m'].transform(
144     lambda x: x.shift(1).rolling(3, min_periods=
145     3).max()
146     )
147
148     # Pressure trends (not target component) -
149     # current minus past
150     df['pressure_msl_trend_3h'] = (
151     df['pressure_msl'] - df.groupby('location')['
152     pressure_msl'].shift(3)
153     )
154
155     df['pressure_msl_trend_6h'] = (
156     df['pressure_msl'] - df.groupby('
157     location')['pressure_msl'].shift(6)

```

```

143     )
144
145     # Build up signals: humidity and low-cloud lags
146     df['cloud_cover_low_lag1'] = df.groupby('
        location')['cloud_cover_low'].shift(1)
147     df['humidity_lag1'] = df.groupby('location')['
        relative_humidity_2m'].shift(1)
148     df['humidity_lag3'] = df.groupby('location')['
        relative_humidity_2m'].shift(3)
149
150     return df
151
152
153 def build_target(
154     df: pd.DataFrame,
155     light_precip_mm: float =
        WET_VERANDA_LIGHT_PRECIP_MM,
156     gust_kmh: float = WET_VERANDA_GUST_KMH,
157     heavy_precip_mm: float =
        WET_VERANDA_HEAVY_PRECIP_MM,
158 ) -> pd.DataFrame:
159     """Construct the binary `wet_veranda` target.
160
161     A wet-veranda event is defined as either:
162     - light-or-heavier rain combined with
meaningful wind gusts, OR
163     - rainfall above the heavy-rain threshold
regardless of wind.
164
165     Defaults reflect the provisional thresholds
documented in EDA Section 12.
166
167     Returns a copy of df with an int-typed `
wet_veranda` column appended.
168     """
169     df = df.copy()
170     df["wet_veranda"] = (
171         ((df["precipitation"] > light_precip_mm) & (
            df["wind_gusts_10m"] > gust_kmh))
172         | (df["precipitation"] > heavy_precip_mm)
173     ).astype(int)

```

```

174     return df
175
176
177
178 def save_processed(df: pd.DataFrame, output_path:
    Path, stages: list) -> Path:
179     """Write the processed dataset plus a
        reproducibility manifest."""
180     df.to_parquet(output_path, index=False)
181
182     with open(output_path, "rb") as f:
183         file_hash = hashlib.sha256(f.read()).
            hexdigest()
184
185     manifest = {
186         "data_file": output_path.name,
187         "file_sha256": file_hash,
188         "file_size_bytes": output_path.stat().
            st_size,
189         "built_at_utc": datetime.now(timezone.utc).
            isoformat(),
190         "source_files": [WEATHER_FILE, ONI_FILE],
191         "row_count": len(df),
192         "columns": list(df.columns),
193         "pipeline_stages": stages,
194         "target_thresholds": {
195             "light_precip_mm":
                WET_VERANDA_LIGHT_PRECIP_MM,
196             "gust_kmh": WET_VERANDA_GUST_KMH,
197             "heavy_precip_mm":
                WET_VERANDA_HEAVY_PRECIP_MM,
198         },
199     }
200     manifest_path = output_path.with_suffix(".
        manifest.json")
201     with open(manifest_path, "w") as f:
202         json.dump(manifest, f, indent=2)
203
204     return manifest_path
205
206

```

```

207 if __name__ == "__main__":
208     PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
209     stages = []
210
211     print("Loading raw data...")
212     weather, oni = load_raw_data()
213     stages += ["load", "drop_boundary_layer_height"
214 , "drop_rain"]
215     print(f" Weather: {weather.shape}, ONI: {oni.
216 shape}")
217
218     print("Merging ONI...")
219     df = merge_oni(weather, oni)
220     stages.append("merge_oni")
221     missing = df["oni"].isnull().sum()
222     print(f" Missing ONI after merge+ffill: {
223 missing}")
224
225     print("Adding temporal features...")
226     df = add_temporal_features(df)
227     stages.append("temporal_features")
228
229     print("Adding lag features...")
230     df = add_lag_features(df)
231     stages.append("lag_features")
232
233     print("Building target...")
234     df = build_target(df)
235     stages.append("build_target")
236     pos_rate = df["wet_veranda"].mean()
237     print(f" Thresholds: light_precip={
238 WET_VERANDA_LIGHT_PRECIP_MM} mm, "
239 f"gust={WET_VERANDA_GUST_KMH} km/h,
240 heavy_precip={WET_VERANDA_HEAVY_PRECIP_MM} mm")
241     print(f" Positive class rate: {pos_rate * 100:.
242 2f}% ({int(df['wet_veranda'].sum()):,} events)")
243
244     output_path = PROCESSED_DIR / OUTPUT_FILE
245     manifest_path = save_processed(df, output_path,
246 stages)
247

```

```
241     print(f"\nSaved {len(df):,} rows to {output_path  
    .name}")  
242     print(f"Manifest: {manifest_path.name}")  
243     print(f"Columns ({df.shape[1]}): {list(df.  
    columns)}")
```